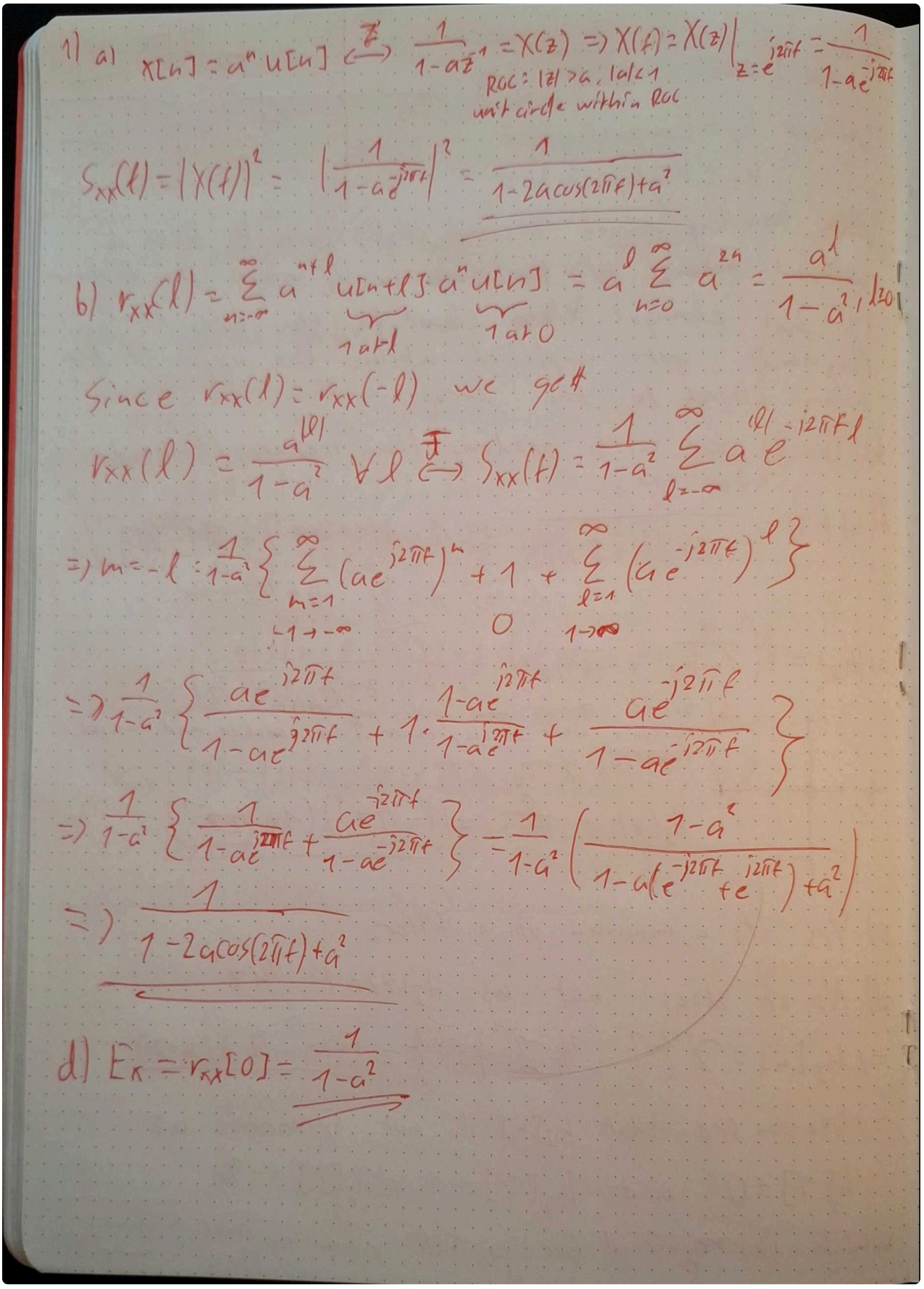
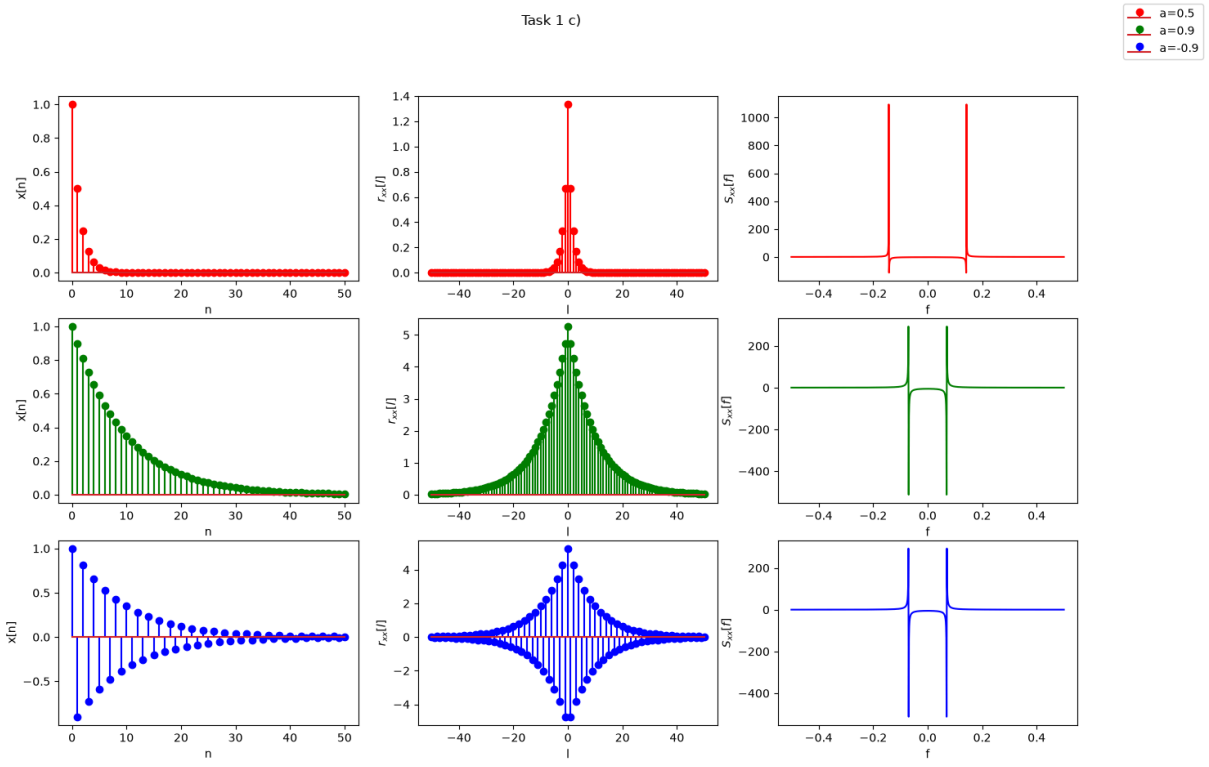
Problem 1)

a), b) & d)



c)

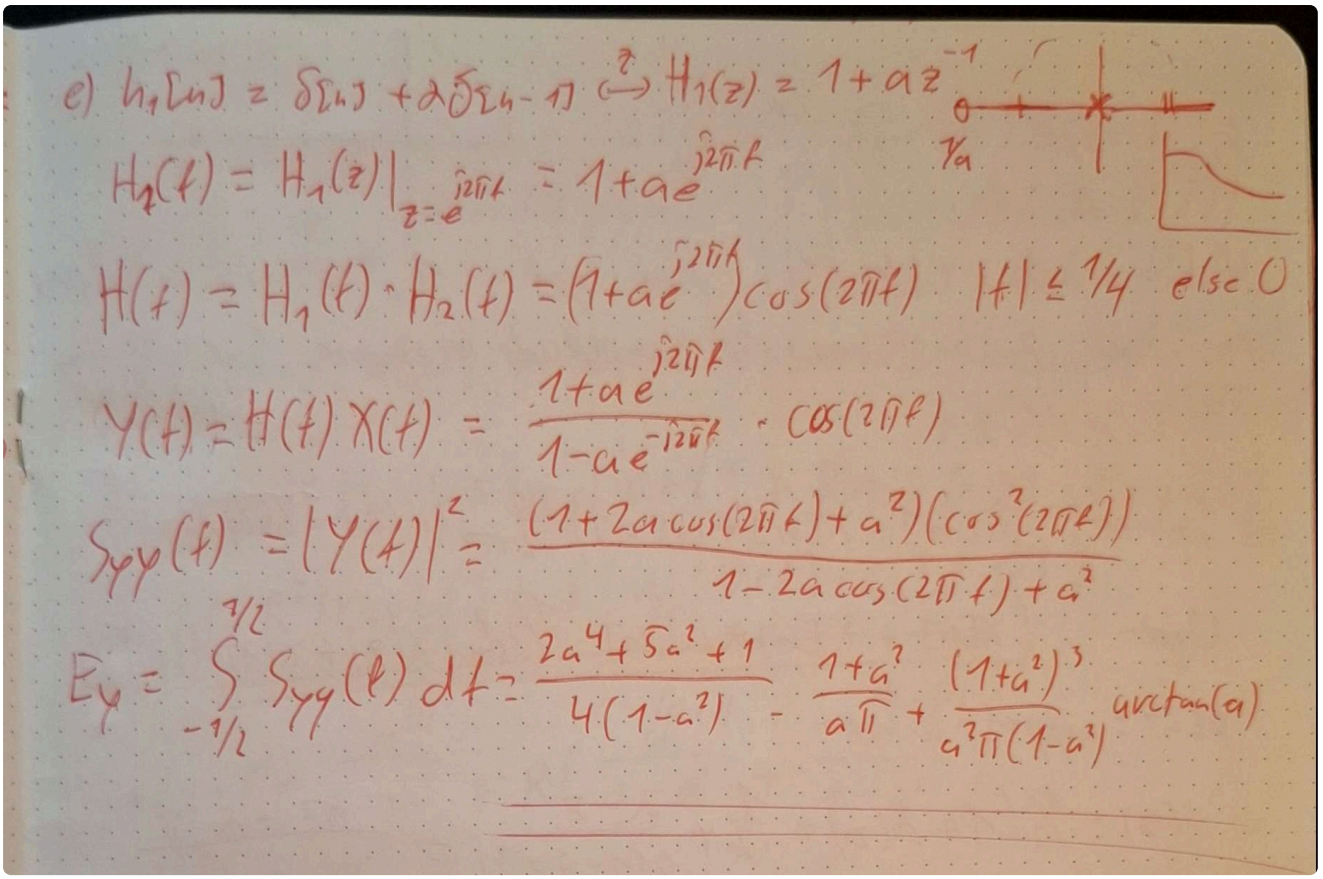


As expected, the higher value of a for the green plot means a longer time until the signal dies out compared to the red one, while the blue matches the green except flipping every other n value.

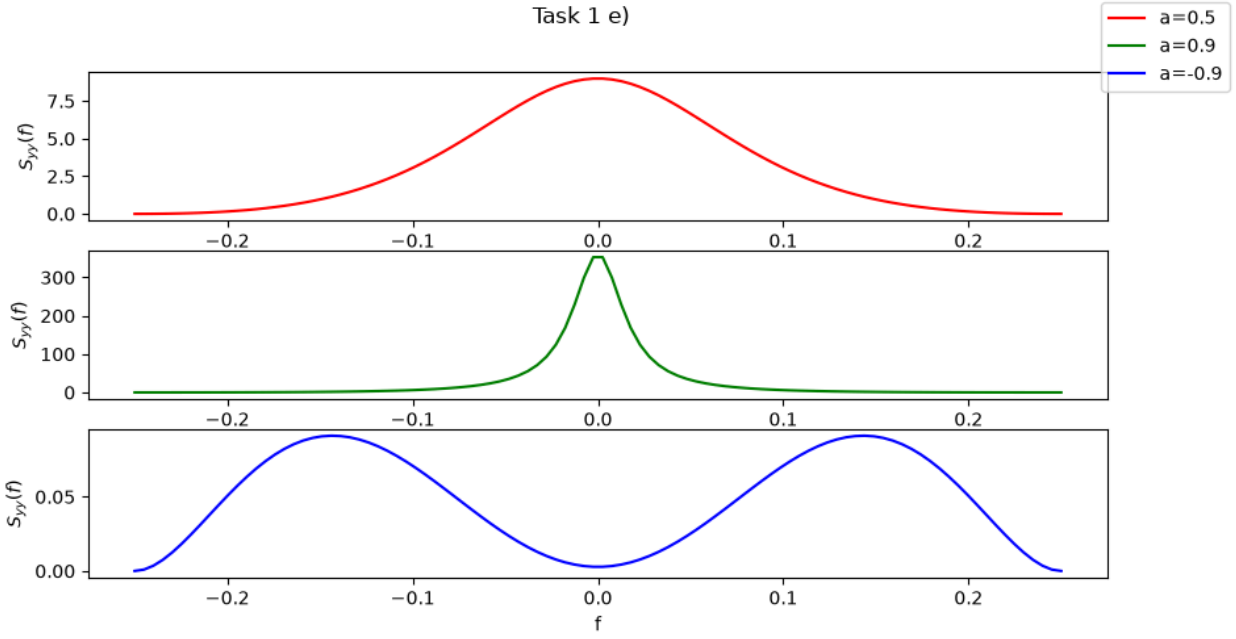
The autocrossrelation in the middle shows that for $r_{xx}[0]$ they are all ofcourse very similar, and again the higher the a value the more similar the crossrelation as we add delay becomes, as for low a the signal quickly dies out. Again the blue matches the green more in shape, but is mirrored and will have negative autocrossrelation for certain delays l.

Finally the energy density spectrum is purely dependant on frequency and $|a|$ so the blue and green are identical and the red peaks higher for its given energy is in its larger frequency components.

e)



Plotting $S_{yy}(f)$:



Code:

PYTHON

```python
import matplotlib
matplotlib.use("QtAgg")

import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import freqz, correlate

# c)

n = np.arange(0,51)

a1 = 0.5
a2 = 0.9
a3 = -0.9
a = [a1,a2,a3]

x = [a[0]**n, a[1]**n, a[2]**n]

l = np.arange(-50, 51)

r_xx = [a[0]**abs(l)/(1-a[0]**2),
        a[1]**abs(l)/(1-a[1]**2),
        a[2]**abs(l)/(1-a[2]**2),]

f = np.linspace(-0.5, 0.5, 1000)

S_xx = [1/(1 - 2 * np.cos(2 * np.pi * f) + a[0]**2),
        1/(1 - 2 * np.cos(2 * np.pi * f) + a[1]**2),
        1/(1 - 2 * np.cos(2 * np.pi * f) + a[2]**2),]

fig, (ax1, ax2, ax3) = plt.subplots(3,3, figsize=(15,10))

ax1[0].stem(n, x[0], linefmt="r", label=f"a={a[0]}")
ax2[0].stem(n, x[1], linefmt="g", label=f"a={a[1]}")
ax3[0].stem(n, x[2], linefmt="b", label=f"a={a[2]}")

ax1[0].set_ylabel("x[n]")
ax2[0].set_ylabel("x[n]")
ax3[0].set_ylabel("x[n]")

ax1[0].set_xlabel("n")
ax2[0].set_xlabel("n")
ax3[0].set_xlabel("n")


ax1[1].stem(l, r_xx[0], linefmt="r", )
ax2[1].stem(l, r_xx[1], linefmt="g", )
ax3[1].stem(l, r_xx[2], linefmt="b", )

ax1[1].set_ylabel(r"$r_{xx}[l]$")
ax2[1].set_ylabel(r"$r_{xx}[l]$")
ax3[1].set_ylabel(r"$r_{xx}[l]$")
```

```
ax1[1].set_xlabel("l")
ax2[1].set_xlabel("l")
ax3[1].set_xlabel("l")


ax1[2].plot(f, S_xx[0], c="r", )
ax2[2].plot(f, S_xx[1], c="g", )
ax3[2].plot(f, S_xx[2], c="b", )

ax1[2].set_ylabel(r"$S_{xx}[f]$")
ax2[2].set_ylabel(r"$S_{xx}[f]$")
ax3[2].set_ylabel(r"$S_{xx}[f]$")

ax1[2].set_xlabel("f")
ax2[2].set_xlabel("f")
ax3[2].set_xlabel("f")




fig.suptitle("Task 1 c)")
fig.legend()


f_4 = np.linspace(-1/4, 1/4, 100)

S_yy_1 = ( (1 + 2*a[0]*np.cos(2*np.pi*f_4) + a[0]**2) *
np.cos(2*np.pi*f_4)**2 ) / ( 1 - 2*a[0]*np.cos(2*np.pi*f_4) + a[0]**2)

S_yy_2 = ( (1 + 2*a[1]*np.cos(2*np.pi*f_4) + a[1]**2) *
np.cos(2*np.pi*f_4)**2 ) / ( 1 - 2*a[1]*np.cos(2*np.pi*f_4) + a[1]**2)

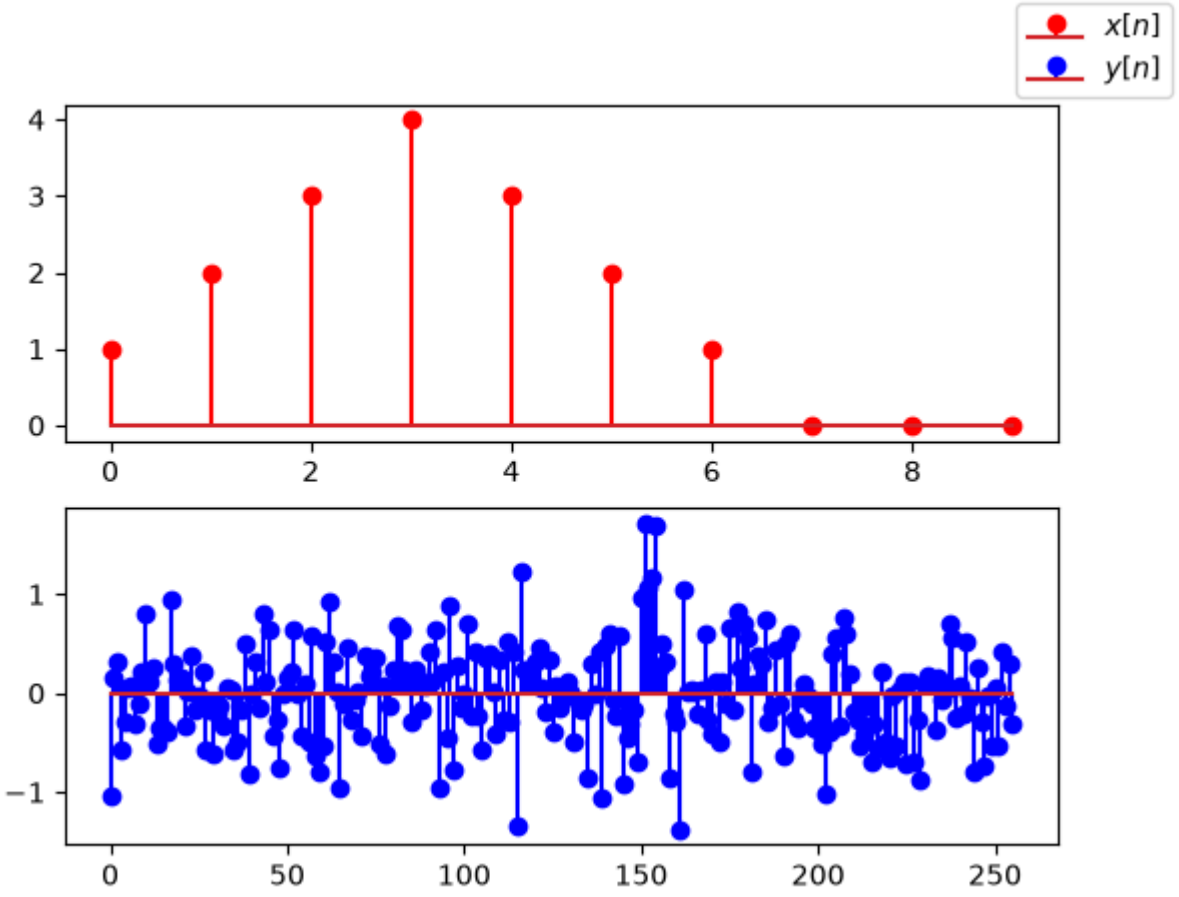
S_yy_3 = ( (1 + 2*a[2]*np.cos(2*np.pi*f_4) + a[2]**2) *
np.cos(2*np.pi*f_4)**2 ) / ( 1 - 2*a[2]*np.cos(2*np.pi*f_4) + a[2]**2)


fig2, ax = plt.subplots(3,1, figsize=(10,5))
ax[0].plot(f_4, S_yy_1, c="r", label=f"a={a[0]}")
ax[0].set_xlabel("f")
ax[0].set_ylabel(r"$S_{yy}(f)$")
ax[1].plot(f_4, S_yy_2, c="g", label=f"a={a[1]}")
ax[1].set_xlabel("f")
ax[1].set_ylabel(r"$S_{yy}(f)$")
ax[2].plot(f_4, S_yy_3, c="b", label=f"a={a[2]}")
ax[2].set_xlabel("f")
ax[2].set_ylabel(r"$S_{yy}(f)$")
fig2.legend()
fig2.suptitle("Task 1 e)")
plt.show()
```



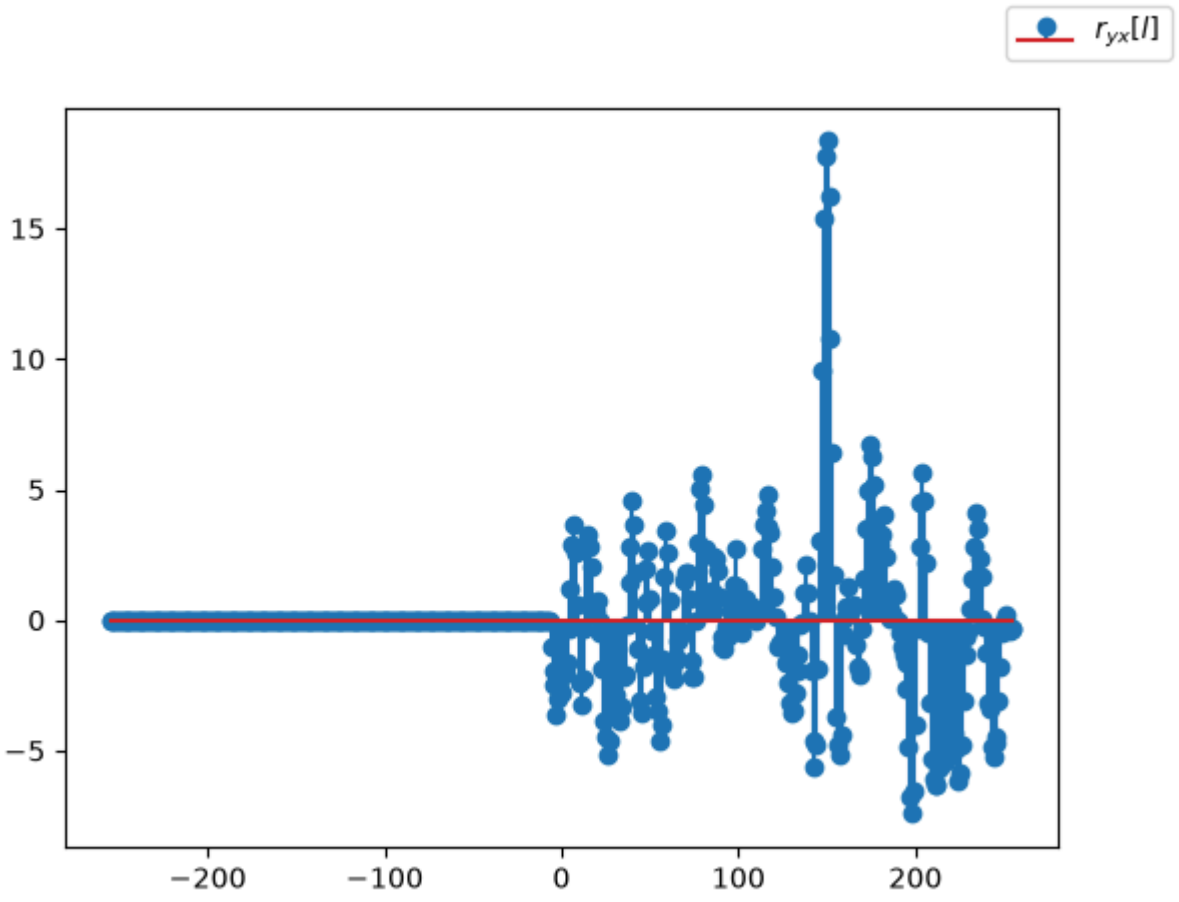
Problem 2)

a)



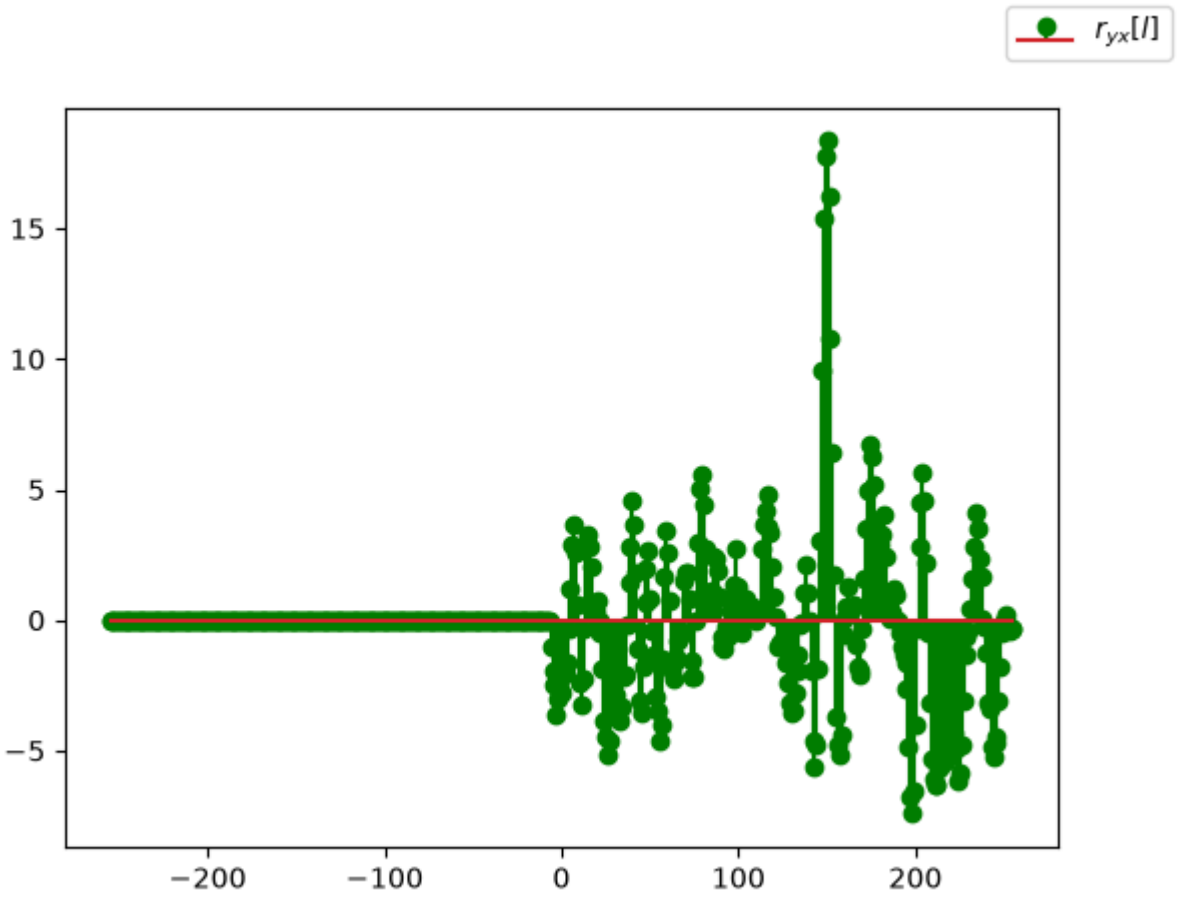
Cant really reliably tell wether has hit anything from within all the noice, but if it did it looks like around 150 steps it had a max peak which could indicate the top of the sent x[n] beeing returned with the delay and scaled down roughly 1/4.

b)



l goes from -255 to but not including 256

c)



Its the same as b) which im hoping the problem ment cuz nothing else makes sense! (but in green!)

d)

Printing out the maximum value and geting the index for it, we get that the crosscorrelation peaks at around $l = 150$ which seems to match what looked to be when the returned signal came back from a), meaning $D = 150$. This is alot more clear compared to looking at the raw signal however.

Code:

PYTHON

```python
import matplotlib
matplotlib.use("QtAgg")

from scipy.fft import fft
from scipy.signal import freqz, lfilter, convolve, correlate
from scipy.io import loadmat
import matplotlib.pyplot as plt
import numpy as np

# a)
mat_dict = loadmat("signals.mat")

x = mat_dict["x"][0]

y = mat_dict["y"][0]

n = np.arange(0, len(x))

fig_a, (ax1_a, ax2_a) = plt.subplots(2, 1)

ax1_a.stem(n[0:10], x[0:10], label=r"$x[n]$", linefmt="r")
ax2_a.stem(n, y, label=r"$y[n]$", linefmt="b")

fig_a.legend()
#plt.savefig("task_2_a")

# b)
r_yx = correlate(y,x)

l = np.arange(-255, 256)

fig_b, ax_b = plt.subplots(1)
ax_b.stem(l, r_yx, label=r"$r_{yx}[l]$")
fig_b.legend()
#plt.savefig("task_2_b")


# c)

r_yx_v2 = np.convolve(y, np.flip(x))

fig_c, ax_c = plt.subplots(1)
ax_c.stem(l, r_yx_v2, label=r"$r_{yx}[l]$", linefmt="g")
fig_c.legend()
plt.savefig("task_2_c")

# d)
print(r"Max of r_{yx}[l] is:")
print(f"{np.argmax(r_yx_v2) - 255}")

plt.show()
```

Problem 3)

a)

$$y[n] = x[n] + \alpha \cdot x[n-R] \implies Y(z) = X(z) + \alpha X(z) z^{-R}$$

$$H(z) = 1 + \alpha z^{-R}$$

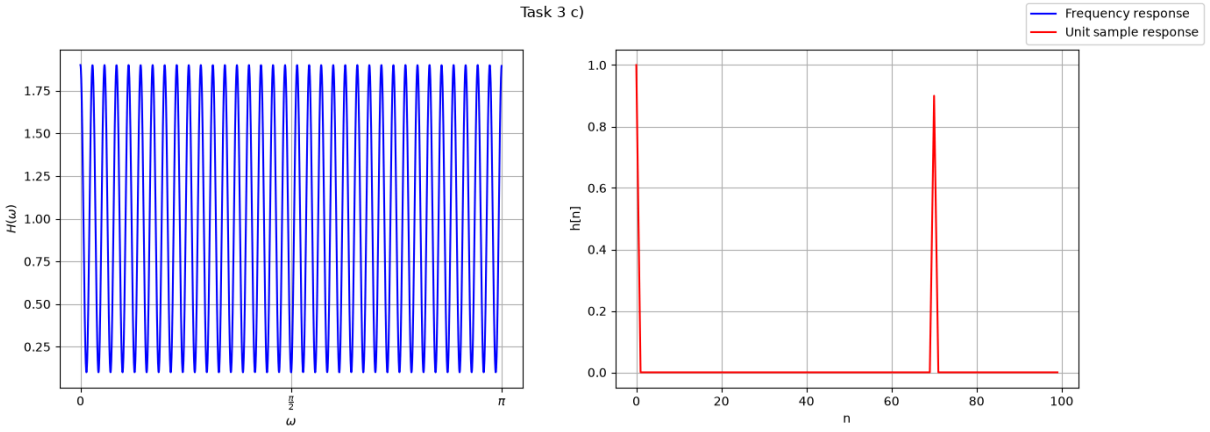
$$h[n] = \delta[n] + \alpha \delta[n-R]$$

b)

Guessing task meant 22.05kHz. This means that each n is $\frac{1}{22050}$ of a second and a delay of $R \implies \frac{R}{22050}$ seconds.

c)

$R = 70$ and $\alpha = 0.9$



d)

The larger R the more delay it is before the echo comes given as in b), and alpha decides how loud the echo is, pretty funny to do around $R = 10_000$ and $\alpha = 0.9$, but horrible to listen to.

e)

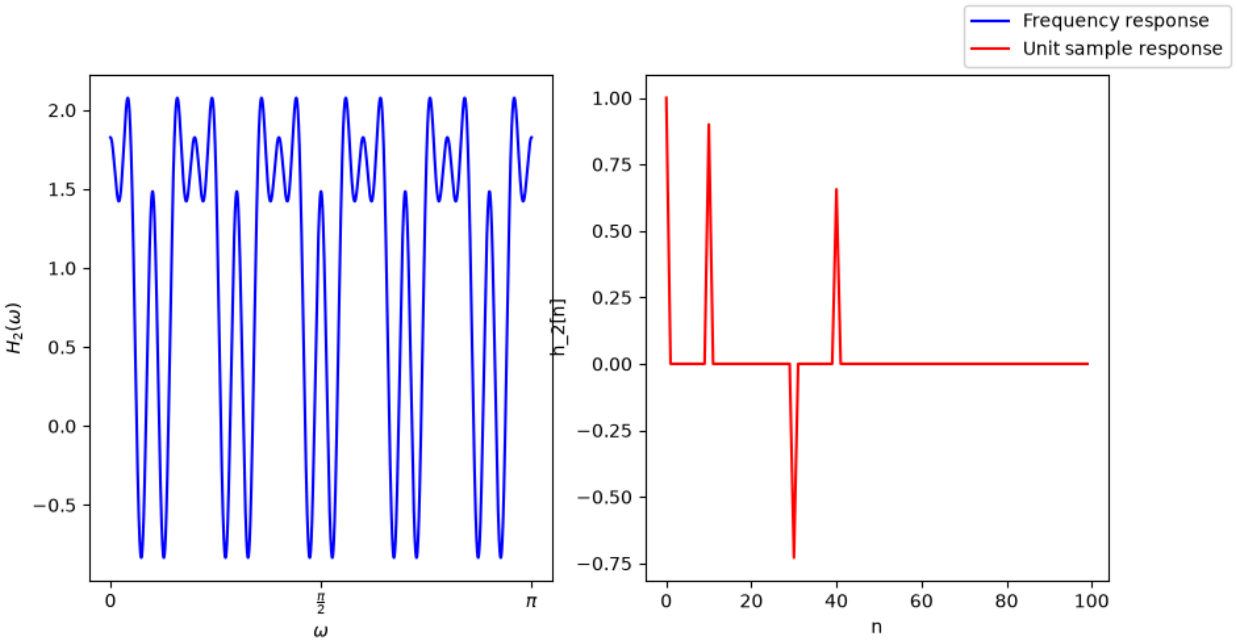
$$E(Z) = X(Z) + \alpha z^{-R} X(Z) = (1 + \alpha z^{-R}) X(Z)$$

$$Y(Z) = E(Z) - \alpha^N z^{-R} z^{-(N-1)R} E(Z) = (1 - \alpha^N z^{-R} z^{-(N-1)R}) E(Z)$$

$$H(z) = (1 - \alpha^N z^{-R} z^{-(N-1)R})(1 + \alpha z^{-R}) = 1 - \alpha^N z^{-R-(N-1)R} + \alpha z^{-R} + \alpha^{N+1} z^{-2R-(N-1)R}$$

See code at end for filter

f)



It has more spikes now, some are upside down aswell! This is for R = 10 and N=3 with alpha=0.9

g)

With the right values for R and N, i found around 1000 and 10 with alpha=0.5, it sounds like propperish echo. Like standing in a church and playing piano.

h)

The second one sounds most natural, as in a room there will be multiple echoes at different delays and amplitudes, unless theres no echo ofc.

Code:

PYTHON

```python
import matplotlib
matplotlib.use("QtAgg")

import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import freqz, dlti, dimpulse, lfilter
import sounddevice as sd
from scipy.io.wavfile import read


alpha = 0.9
R = 10

fs, x = read("piano.wav")
print(f"fs: {fs}")

x_scaled = x / np.max(x)
#sd.play(x_scaled, fs)
#input("Playing original")

b = np.zeros(R+1)
b[0] = 1
b[R] = alpha
a = np.zeros(R+1)
a[0] = 1



w, h_w = freqz(b, a, 22050)

filter = dlti(b,a)

t, y_out = dimpulse(filter)
y_impulse = y_out[0].flatten()

y = lfilter(b, a, x)
y_scaled = y / np.max(y)
#sd.play(y_scaled, fs)
#input("Playing filtered")

tick_positions = [0, np.pi/2, np.pi]
tick_labels = [r'$0$', r'$\frac{\pi}{2}$', r'$\pi$']
fig, (ax1, ax2) = plt.subplots(1,2, figsize=(15,5))


ax1.plot(w, h_w, label="Frequency response", c="b")
ax1.set_xlabel(r"$\omega$")
ax1.set_ylabel(r"$H(\omega)$")
ax1.set_xticks(tick_positions)
ax1.set_xticklabels(tick_labels)
ax1.grid()
```

```
ax2.plot(t, y_impulse, label="Unit sample response", c="r")
ax2.set_xlabel("n")
ax2.set_ylabel("h[n]")
ax2.grid()



fig.legend()
fig.suptitle("Task 3 c)")

print("Test")
N = 3

e1 = abs(-R-(N-1)*R)
e2 = abs(-2*R-(N-1)*R)

b2 = np.zeros(e2+1)
a2 = np.zeros(e2+1)
#b2 = np.zeros(abs(-2*R-(N-1)*R)+1)
#a2 = np.zeros(abs(-2*R-(N-1)*R)+1)
a2[0] = 1
b2[0] = 1
b2[R] = alpha
b2[e1] = -alpha**N
b2[e2] = alpha**(N+1)

y2 = lfilter(b2, a2, x)
y2_scaled = y2 / np.max(y2)
#sd.play(y2_scaled, fs)
#input("Playing second filter")



w2, h_w2 = freqz(b2, a2, 22050)
filter2 = dlti(b2, a2)
t2, y2_out = dimpulse(filter2)
y2_impulse = y2_out[0].flatten()

print(len(y2_impulse))
print(len(t2))

fig2, (ax3, ax4) = plt.subplots(1,2, figsize = (10,5))
ax3.plot(w2, h_w2, label="Frequency response", c="b")
ax3.set_xlabel(r"$\omega$")
ax3.set_ylabel(r"$H_2(\omega)$")
ax3.set_xticks(tick_positions)
ax3.set_xticklabels(tick_labels)
ax3.grid()

ax4.plot(t2, y2_impulse, label="Unit sample response", c="r")
ax4.set_xlabel("n")
ax4.set_ylabel(r"h_2[n]")
ax4.grid()
```

```
fig2.legend()
ax3.grid()
ax4.grid()


print("Exiting")
plt.show()
```